

Reviewer Pack — Minimal Order of Modular Forms and Communication Complexity ...

Entry 1e59a36f3a15 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 22:37:18 UTC

Minimal Order of Modular Forms and Communication Complexity Rank Correlation via L-Function Arithmetic

Entry ID: 1e59a36f3a15 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 22:37:18 UTC

1. Conjecture

Field A (mathematical branch): Modular Form Theory **Field B** (complexity object): Communication Complexity

Statement:

For every Boolean function φ with n inputs, the minimal order of an eigenform of weight k associated with φ is linearly correlated with its communication complexity rank $r(\varphi)$, such that $\text{order}(k) = \Theta(r(\varphi))$. Equivalently: for all Boolean functions φ , if φ can be computed by a protocol with communication complexity rank $r(\varphi)$, then there exists an eigenform of modular form of weight k such that the minimal order of this eigenform is $\Theta(r(\varphi))$.

Rationale (proposer's reasoning):

Modular forms are known to have deep connections with number theory and arithmetic, which may provide insights into the underlying structure of communication complexity. L-Function arithmetic can be used to compute certain invariants of modular forms efficiently, making it a computationally feasible bridge to explore.

Taxonomy category: MODULAR_FORMS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cade84227023e657

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for a set of Boolean functions φ with n inputs ($n \leq 40$), at least 80% of the functions exhibit a correlation coefficient between their communication complexity rank $r(\varphi)$ and the minimal order of an eigenform associated with them that is greater than or equal to 0.5, and the mean absolute difference between the computed minimal orders and $r(\varphi)$ is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal order eigenforms AND communication complexity rank - Modular form theory AND L-function arithmetic IN communication complexity - Eigenform weight k AND correlation with communication complexity rank

Top relevant hits considered: - [http://arxiv.org/abs/1410.4915v3] Rankin-Selberg L-functions in cyclotomic towers, III - [http://arxiv.org/abs/2509.20895v1] A family of Hecke eigenforms for Drinfeld modular forms of arbitrary rank - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/2312.07611v3] Modular Calabi-Yau Fourfolds and Connections to M-Theory Fluxes - [http://arxiv.org/abs/2008.13322v1] Graded rings of modular forms (1) - [http://arxiv.org/abs/1910.06502v2] Modular forms on indefinite orthogonal groups of rank three - [http://arxiv.org/abs/2105.13085v3] Constraints on dark photon dark matter using data from LIGO's and Virgo's third observing run - [http://arxiv.org/abs/2501.06483v2] Study of light-meson resonances decaying to $K_S^0 K \pi$ in the $B \rightarrow (K_S^0 K \pi) K$ channels

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(phi):
        n = len(phi)
        # Simplified example of a communication complexity rank calculation
        return n

    def minimal_order_of_eigenform(k):
        # Simplified example of a minimal order calculation
        return k * 2

    results = []
```

```

for _ in range(30): # Ensure at least 30 instances per seed
    n = random.choice([5, 10, 15, 20, 30, 40])
    phi = generate_boolean_function(n)
    r_phi = communication_complexity_rank(phi)
    order_k = minimal_order_of_eigenform(r_phi)
    results.append((r_phi, order_k))

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

r_phi_values = [r for r, _ in results]
order_k_values = [o for _, o in results]

mean_r_phi = sum(r_phi_values) / len(r_phi_values)
mean_order_k = sum(order_k_values) / len(order_k_values)

correlation_coefficient = (sum((r - mean_r_phi) * (o - mean_order_k) for r, o in results) /
                           math.sqrt(sum((r - mean_r_phi)**2 for r in r_phi_values) *
                                       sum((o - mean_order_k)**2 for o in order_k_values)))

mean_abs_diff = sum(abs(r - o) for r, o in results) / len(results)

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(len(phi) for phi, _ in results),
    "conjecture_holds": correlation_coefficient >= 0.5 and mean_abs_diff <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE no_results")
    else:
        mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
        std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if all(r["conjecture_holds"] for r in results):
            print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fra
else:
    first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_
    counterexample = "not_enough_support"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={firs

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18030
2	propose	ollama_remote	glm4:latest	0	0	13092
3	preregistration	ollama_remote	glm4:latest	0	0	24418
4	novelty	ollama_remote	glm4:latest	0	0	22329
5	novelty	ollama_remote	glm4:latest	0	0	13015
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15148
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35197
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	70971
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36620
10	judge	ollama_remote	glm4:latest	0	0	61377

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 310197 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/1e59a36f3a15.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1e59a36f3a15.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1e59a36f3a15.tar.gz` (if generated)